

Endless Engine

Türkçe Kullanım Kılavuzu

com.endlessengine.idle v1.1.0

Unity 6.3 LTS

Unity ile idle / incremental oyun geliştiren C# geliştiricileri

Son Güncelleme: 2026-04-28

50 bölümlük, kurulumdan çalışan oyuna kadar uçtan uca kullanım kılavuzu.

İçindekiler

1. Kurulum ve Gereksinimler	10
Bağımlılıklar	10
Paket Kurulumu	10
Hızlı Test: MinimalIdle Örneği	10
2. Temel Mimari	11
Genel Akış	11
Beş Temel Kural	11
Namespace Haritası	11
3. Hızlı Başlangıç	12
Adım 1 — Config Asset'leri Oluşturun	12
Adım 2 — Sahne Hiyerarşisi	12
Adım 3 — Bootstrap Script'i	12
Adım 4 — UI'dan Jeneratör Satın Alma	12
4. ConfigRegistry	14
Erişim	14
Prodüksiyon: Addressables ile Yükleme	14
Geliştirme / Test: Doğrudan Enjeksiyon	14
Realm Değiştirme	14
Önemli Kurallar	14
5. SaveService	15
Config: SchemaVersionSO	15
Kullanım	15
Kendi Sisteminizi Kayıt Etme	15
Kayıt Dosya Konumu	15
SaveProviderOrder Sabitler Listesi	15
Şema Migrasyonu	16
6. EconomyService	17
Config: EconomyConfigSO	17
Başlatma	17

API	17
Olaylar	17
UI Bağlantısı	17
7. TickEngine	18
Inspector Ayarları	18
Kullanım	18
Neden Her Şey TickEngine'e Bağlı?	18
8. GeneratorSystem	19
Config: GeneratorConfigSO	19
Başlatma	19
API	19
Toplu Satın Alma	19
Gelir Formülü	19
9. PassiveIncomeService	20
Başlatma	20
10. OfflineProgressService	21
EconomyConfigSO İlgili Alanlar	21
Başlatma	21
UI'da Gösterme	21
11. ClickYieldService	22
Başlatma	22
API	22
12. ClickLoopService	23
Sistem Bileşenleri	23
Config: ClickLoopConfigSO	23
Config: ClickTargetConfigSO	23
ClickTarget Sahneye Ekleme	23
Toplu Yerleştirme: ClickTargetSpawner	24
Başlatma	24
Olaylar	24
HUD Bağlantısı	24

Offline Hesaplama	25
Upgrade Bağlantıları (StatType Enum)	25
Kayıt / Yükleme	25
13. HarvestLoopService	26
Sistem Bileşenleri	26
Config: HarvestAreaConfigSO	26
Config: HarvestNodeConfigSO	26
HarvestNode Sahneye Ekleme	26
HarvestCursor Kurulumu	27
Başlatma	27
Olaylar	27
HUD Bağlantısı	27
Offline Hesaplama	28
Upgrade Bağlantıları (StatType Enum)	28
Kayıt / Yükleme	28
ClickLoop ile HarvestLoop Karşılaştırması	28
14. UpgradeTreeService	29
Config: UpgradeNodeConfigSO	29
Tüm StatType Değerleri	29
Başlatma	29
API	30
Görsel Düzenleme	30
15. UpgradeApplicationSystem	31
Stat Formülü	31
API	31
16. SkillTreeService	32
Config: SkillTreeConfigSO	32
Başlatma	32
API	32
Görsel Düzenleme	32
17. ResearchService	33

Config: ResearchTreeConfigSO ve ResearchNodeConfigSO	33
Başlatma	33
API	33
18. PrestigeStateManager	34
Config: PrestigeConfigSO	34
Başlatma	34
Prestige Akışı	34
API	34
19. AscensionStateManager	35
Config: AscensionLayerConfigSO	35
Başlatma	35
API	35
20. WaveSpawnManager	36
Config: WaveConfigSO	36
Başlatma	36
API	36
21. AutoBattleController	37
Config: PlayerBaseStatConfigSO	37
Başlatma	37
22. DamageSystem	38
Dinleme	38
23. InputProvider	39
IInputProvider Arayüzü	39
Kurulum	39
Kural: Tek Dosya	39
24. BuildingService	40
Config: BuildingConfigSO	40
Kullanım	40
25. PetService	41
Config: PetConfigSO	41
Başlatma	41

API	41
26. EventService	42
Kullanım	42
27. ChallengeService	43
Config: ChallengeConfigSO	43
Başlatma	43
API	43
28. MilestoneTracker	44
Config: MilestoneConfigSO	44
Başlatma	44
API	44
29. QuestService	45
Kullanım	45
30. MinigameService	46
Başlatma	46
API	46
31. MergeService	47
Config: MergeConfigSO	47
API	47
32. ConversionService	48
Config: ConversionDatabaseSO	48
API	48
33. TraitService	49
API	49
34. UnlockLogService	50
Başlatma	50
API	50
35. StatisticsService	51
Başlatma	51
API	51
StatDefinitionSO	51

36. NotificationService	52
API	52
37. LeaderboardService	53
API	53
38. TutorialService	54
API	54
39. RealmSystem	55
Yeni Realm Oluşturma	55
Realm Değiştirme	55
40. AudioService	56
API	56
41. BigDouble	57
Ne Zaman Kullanılır?	57
Aktifleştirme	57
Yapısı	57
Dikkat Edilecekler	57
42. Editor Araçları	58
New Game Wizard Oyun Tipleri	58
43. Test Stratejisi	59
Katman 1 — Unit Tests	59
Katman 2 — Integration Tests	59
Katman 3 — Performance Tests	59
Test Çalıştırma	59
Önemli Test Kalıpları	59
44. Tarif 1: Klasik Idle	61
Kullanılan Sistemler	61
Kurulum	61
Bootstrap Script'i	61
Tıklama Butonu	61
Jeneratör UI	61
Offline Popup	62

45. Tarif 2: Aktif Clicker	63
Kullanılan Sistemler	63
Kurulum	63
Sahne Yapısı	63
Config'leri Oluşturun	63
Bootstrap Script'i	64
ClickTarget Sahnede Kurulumu	64
Upgrade Yükseltmeleri	64
UI	64
46. Tarif 3: Hasat Loop	65
Kullanılan Sistemler	65
Sahne Yapısı	65
Config'leri Oluşturun	65
Bootstrap Script'i	66
HarvestNode Sahnede Kurulumu	66
Upgrade Yükseltmeleri	66
47. Tarif 4: Idle-vs	67
Kullanılan Sistemler	67
Ek Config'ler	67
Ek Sistemler Bootstrap'e Ekle	67
Dalga Sonu Upgrade Ekranı	67
Prestige Butonu	67
48. Tarif 5: Merge Idle	68
Kullanılan Sistemler	68
Kilit Adımlar	68
49. Tarif 6: Prestige-Heavy RPG Idle	69
Kullanılan Sistemler	69
Ascension Kurulumu	69
Prestige → Skill Puanı	69
Ascension → Trait Seçimi	69
Araştırma Kuyruğu	69

50. Sorun Giderme**70**

"ConfigNotLoadedException" veya null ref config hatası	70
Testlerde Input System her zaman 0 döndürüyor	70
ClickTarget / HarvestNode Awake'te config null	70
ClickLoopService ve ClickYieldService aynı anda altın veriyor	70
Prestige sonrası stat güncellenmiyor	70
Çevrimdışı hesap çalışmıyor	70
BigDouble kayıt/yükleme uyumsuzluğu	71
HarvestNode veya ClickTarget kayıt sonrası senkronize değil	71

Bu kılavuz Endless Engine'i sıfırdan kurarak tam çalışan bir idle oyunu yapmanız için gereken her şeyi kapsar. İlk okumada baştan sona geçin; sonra ihtiyaç duydukça ilgili bölüme dönün.

1. Kurulum ve Gereksinimler

Bağımlılıklar

Aşağıdaki Unity paketleri kurulu olmalıdır (Window → Package Manager):

Paket	Minimum Versiyon	Not
Input System	1.14.2	Kurulum sonrası Unity yeniden başlatma ister — "Yes" deyin
Addressables	2.7.6	Config yükleme için
Newtonsoft Json	3.2.2	Kayıt serileştirme için

Paket Kurulumu

1. `com.endlessengine.idle/` klasörünü projenizin Packages/ klasörüne kopyalayın:

```
YourProject/  
└─ Packages/  
    └─ com.endlessengine.idle/  
        └─ Runtime/  
        └─ Editor/  
        └─ Tests/  
        └─ package.json
```

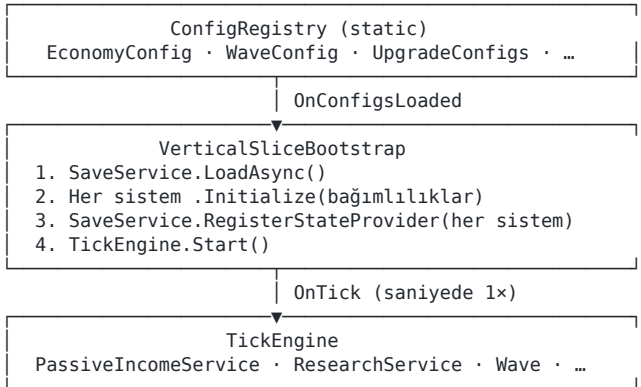
2. Unity editörünü açın — paket otomatik yüklenir.
3. Packages/manifest.json dosyasında "testables" dizisine "`com.unity.inputsystem`" eklendiğinden emin olun (Input System testleri için).

Hızlı Test: MinimalIdle Örneği

Window → Package Manager → Endless Engine → Samples → MinimalIdle → Import
Assets/Samples/.../MinimalIdle/Scenes/MinimalIdle.unity → Aç → Play

2. Temel Mimari

Genel Akış



Beş Temel Kural

Kural	Neden
Config = ScriptableObject	Kodda sihirli sayı yok; tasarımcı Inspector'dan değiştirebilir
Haberleşme = C# event	Sistemler birbirini doğrudan çağırır; bağımlılık azalır
Kayıt = ISaveServiceProvider	Her sistem kendi verisini bildirir; SaveService toplar
Bağımlılık = Initialize() enjeksiyonu	new/singleton yok; birim testleri yazılabilir
Config erişimi = ConfigRegistry	Tek erişim noktası; Addressables veya test inject ile değiştirilebilir

Namespace Haritası

EndlessEngine.Config	ConfigRegistry, tüm *ConfigSO'lar
EndlessEngine.SaveAndLoad	SaveService, ISaveServiceProvider, SaveData, SaveConstants
EndlessEngine.Economy	EconomyService, CurrencyService, InventoryService
EndlessEngine.Economy.Math	IBigInteger, BigInteger, DoubleNumber
EndlessEngine.Flow	TickEngine, PassiveIncomeService, GameFlowStateMachine
EndlessEngine.GeneratorSystem	GeneratorSystem
EndlessEngine.Upgrade	UpgradeTreeService, SkillTreeService, UpgradeApplicationSystem
EndlessEngine.Research	ResearchService
EndlessEngine.Prestige	PrestigeStateManager, AscensionStateManager
EndlessEngine.Wave	WaveSpawnManager
EndlessEngine.Combat	AutoBattleController
EndlessEngine.Damage	DamageSystem
EndlessEngine.Health	HealthSystem, PlayerHealthComponent
EndlessEngine.Input	IInputProvider, InputProviderUnity
EndlessEngine.Building	BuildingService
EndlessEngine.Pet	PetService
EndlessEngine.Events	EventService
EndlessEngine.Challenge	ChallengeService
EndlessEngine.Milestone	MilestoneTracker
EndlessEngine.Statistics	StatisticsService
EndlessEngine.Harvest	HarvestLoopService, HarvestCursor, HarvestNode, ...
EndlessEngine.ClickLoop	ClickLoopService, ClickTarget, ...
EndlessEngine.Click	ClickYieldService, CursorYieldService
EndlessEngine.VFX	VFXController
EndlessEngine.UI	HarvestHUDController, ClickLoopHUDController

3. Hızlı Başlangıç

Bu adımlar 15 dakikada çalışan bir idle oyun verir.

Adım 1 — Config Asset'leri Oluşturun

Tools → Endless Engine → New Game Wizard → Pure Idle → Create

Bu işlem Assets/Configs/ altında şunları otomatik oluşturur: EconomyConfig.asset, WaveConfig.asset, SchemaVersion.asset, örnek GeneratorConfig ve UpgradeNodeConfig asset'leri.

Adım 2 — Sahne Hiyerarşisi

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  GeneratorSystem
  PassiveIncomeService
  UpgradeTreeService
```

Adım 3 — Bootstrap Script'i

```
using UnityEngine;
using EndlessEngine.Config;
using EndlessEngine.Economy;
using EndlessEngine.Flow;
using EndlessEngine.Generator;
using EndlessEngine.SaveAndLoad;
using EndlessEngine.Upgrade;

public class GameBootstrap : MonoBehaviour
{
    [SerializeField] private EconomyConfigSO _economyConfig;
    [SerializeField] private SaveService _saveService;
    [SerializeField] private EconomyService _economyService;
    [SerializeField] private TickEngine _tickEngine;
    [SerializeField] private GeneratorSystem _generatorSystem;
    [SerializeField] private PassiveIncomeService _passiveIncome;
    [SerializeField] private UpgradeTreeService _upgradeTree;

    private async void Awake()
    {
        ConfigRegistry.InjectForTesting(economy: _economyConfig);

        await _saveService.LoadAsync();

        _upgradeTree.Initialize();
        _economyService.Initialize(_upgradeTree, _saveService);
        _generatorSystem.Initialize(ConfigRegistry.Generators, _economyService,
            _saveService);
        _passiveIncome.Initialize(_generatorSystem, _economyService, null);

        _saveService.RegisterStateProvider(_economyService);
        _saveService.RegisterStateProvider(_upgradeTree);
        _saveService.RegisterStateProvider(_generatorSystem);
    }
}
```

Adım 4 — UI'dan Jeneratör Satın Alma

```
public class GeneratorButton : MonoBehaviour
{
    [SerializeField] private string _generatorId = "coal-mine";
    [SerializeField] private GeneratorSystem _generators;

    public void OnClick() => _generators.TryPurchase(_generatorId);
}
```

Play tuşuna basın — altın kazanmaya başlar.

4. ConfigRegistry

Ne yapar: Tüm ScriptableObject config'lerine global erişim sağlayan static merkez. Sisteme tüm ayarların tek kapıdan girdiğini garanti eder.

Erişim

```
EconomyConfigSO economy = ConfigRegistry.Economy;  
WaveConfigSO wave = ConfigRegistry.Wave;  
PrestigeConfigSO prestige = ConfigRegistry.Prestige;  
// Diğerleri: Player, Schema, Run, Realm, UpgradeSelection, RealmRegistry
```

Prodüksiyon: Addressables ile Yükleme

Config asset'lerini Addressables grubuna ekleyin, ardından Bootstrap'te:

```
await ConfigLoadingService.LoadAllAsync(addressableLabel: "configs");  
// OnConfigsLoaded event'i tetiklenir → tüm sistemler buraya abone olarak başlar
```

Geliştirme / Test: Doğrudan Enjeksiyon

```
ConfigRegistry.InjectForTesting(  
    economy: myEconomyConfig, // null geçilen alanlar atlanır  
    wave: myWaveConfig  
);  
// Test sonrası mutlaka temizleyin:  
ConfigRegistry.ClearForTesting();
```

Realm Değiştirme

```
await ConfigRegistry.BeginRealmSwapAsync(realmRegistry.GetPack("fire-realm"));  
// OnRealmSwapped tetiklenir – tüm sistemler cache'lerini yeniler
```

Önemli Kurallar

- OnConfigsLoaded tetiklenmeden önce config'lere erişmeyin — Editor build'de exception fırlatır.
- ConfigRegistry.IsLoaded ile yüklenip yüklenmediğini kontrol edebilirsiniz.
- Config değerlerini runtime'da değiştirmeyin; değişim için Realm Swap kullanın.

5. SaveService

Ne yapar: Oyun durumunu JSON olarak diske yazar/okur. Atomic yazma (yedek + rename), otomatik kayıt, şema versiyonlama ve migrasyon destekler.

Config: SchemaVersionSO

Create → Endless Engine → Schema Version

Alan	Açıklama	Başlangıç Değeri
CurrentSchemaVersion	Mevcut kayıt formatı numarası	1
MinimumCompatibleVersion	Uyumlu en düşük sürüm	1

Kullanım

```
// Yükle (Awake'te, async)
await _saveService.LoadAsync();

// Elle kaydet
await _saveService.SaveAsync();

// Otomatik kayıt (her X saniyede)
_saveService.StartAutoSave(intervalSeconds: 30);

// Olaylar
_saveService.OnSaveLoaded += (data, isNew) => { if (isNew) StartTutorial(); };
_saveService.OnSaveCompleted += (ok) => { if (!ok) ShowSaveError(); };
```

Kendi Sisteminizi Kayıt Etme

Her sistem ISaveServiceProvider arayüzünü uygular:

```
public class MySystem : MonoBehaviour, ISaveServiceProvider
{
    public int ProviderOrder => 50; // küçük = önce; Economy=10, Upgrade=20

    public void OnBeforeSave(SaveData data)
    {
        data.MySystemField = _internalValue;
    }

    public void OnAfterLoad(SaveData data)
    {
        _internalValue = data.MySystemField;
    }
}
```

Bootstrap'te kaydedin:

```
_saveService.RegisterServiceProvider(mySystem);
```

Kayıt Dosya Konumu

Windows : %USERPROFILE%\AppData\LocalLow\[Şirket]\[Oyun]\save_slot_0.json
 Android : /data/data/[bundle_id]/files/save_slot_0.json
 iOS : Application.persistentDataPath/save_slot_0.json

SaveProviderOrder Sabitler Listesi

Sistemlerin OnBeforeSave çağrılma sırası (küçük = önce):

Sabit	Değer	Yazan
Economy	10	CurrentResources
Currency	15	CurrencyBalances

Sabit	Değer	Yazan
UpgradeTree	20	UpgradeNodeStates
Ascension	25	AscensionCounts
Prestige	30	PrestigeCount
Inventory	35	InventoryItems
WaveAndCombat	40	WaveNumber
SkillTree	45	UnlockedSkillNodes
Generator	50	GeneratorStates
Building	55	PlacedBuildings
Click	60	ClickState
Pet	65	PetLevels
Zone	70	ZoneStates
UnlockLog	75	UnlockLogEntries
Milestone	80	CompletedMilestones
Statistics	85	StatisticsValues
Harvest	88	HarvestState
ClickLoop	89	ClickLoopState
Research	90	CompletedResearchNodes

Şema Migrasyonu

Yeni alan ekleyip eski kayıtlarla uyumluluğu korumak için:

Tools → Endless Engine → Schema Bump Utility → Bump Version

Bu araç CurrentSchemaVersion'ı artırır ve Assets/Migrations/MigrationV{N}.cs şablonu oluşturur.

6. EconomyService

Ne yapar: Oyunun ana para birimi (Altın) ledger'ı. Tüm kazanma/harcama buradan geçer. Hard cap uygular, çift harcamaya karşı korur.

Config: EconomyConfigSO

Create → Endless Engine → Economy Config

Alan	Açıklama	Örnek
ResourceHardCap	Maksimum bakiye	1e18
StartingGold	Yeni oyun başlangıç altını	0
NumberBackend	DoubleNumber veya BigDouble	DoubleNumber

Başlatma

```
_economyService.Initialize(
    upgradeTreeQuery: _upgradeTree, // IUpgradeTreeQuery
    saveNotifier: _saveService // ISaveNotifier
);
```

API

```
// Ekle
_economyService.AddResources(500.0);

// Çıkar (yetersizse sessizce yapar, log basar)
_economyService.DeductResources(100.0);

// Yükseltme satın al (maliyet UpgradeTree'den otomatik alınır)
_economyService.TryPurchase("dmg-boost-1");

// Bakiye
double balance = _economyService.CurrentResources;
long balLong = _economyService.CurrentResourcesLong;

// Static erişim (UI için kolaylık)
double b = EconomyService.CurrentResourcesStatic;
```

Olaylar

Olay	Parametre	Ne Zaman
OnResourcesChanged	(double current, double delta)	Her mutasyonda
OnUpgradePurchased	(string nodeId, double cost)	Başarılı alımda
OnPurchaseFailed	(string nodeId, double cost, double balance)	Yetersiz bakiyede

UI Bağlantısı

```
void Start()
{
    EconomyService.OnResourcesChanged += (current, _) =>
        goldText.text = FormatGold(current);
}

string FormatGold(double v)
{
    if (v >= 1e9) return $"{v/1e9:F1}B";
    if (v >= 1e6) return $"{v/1e6:F1}M";
    if (v >= 1e3) return $"{v/1e3:F1}K";
    return $"{v:F0}";
}
```

7. TickEngine

Ne yapar: Oyun saatidir. `TickIntervalSeconds` aralıkla `OnTick` event'ini ateşler. Pasif gelir, araştırma sayacı, dalga sistemi gibi tüm zamanlı sistemler buraya abone olur.

Inspector Ayarları

Alan	Açıklama	Varsayılan
<code>TickIntervalSeconds</code>	Kaç saniyede bir tick	1.0
<code>TimeScale</code>	Hız çarpanı	1.0

Kullanım

```
// Tick'e abone ol
_tickEngine.OnTick += HandleTick;

void HandleTick(float deltaTime)
{
    // deltaTime = TickIntervalSeconds × TimeScale
    _myAccumulator += deltaTime;
}

// Durdur / devam ettir
_tickEngine.Pause();
_tickEngine.Resume();

// Hız değiştir (ör. 2× hız boost satın alındı)
_tickEngine.TimeScale = 2.0f;
```

Neden Her Şey TickEngine'e Bağlı?

`Update()` saniyede 60 kez çalışır. Pasif gelir gibi işlemlerin her frame hesaplanması gereksizdir. TickEngine bunları saniyede 1'e indirir; testlerde `TickEngine.SimulateTick()` ile elle tick atılabilir.

8. GeneratorSystem

Ne yapar: Oyuncunun satın alıp geliştirdiği pasif gelir kaynakları (fabrika, maden, çiftlik vb.). Her tick altın üretir.

Config: GeneratorConfigSO

Create → Endless Engine → Generator Config

Alan	Açıklama	Örnek
GeneratorId	Benzersiz ID	"coal-mine"
DisplayName	Gösterim adı	"Kömür Madeni"
BaseYieldPerSecond	Saniye başına temel kazanç	0.5
BaseCost	İlk satın alma fiyatı	10
CostScalingFactor	Her alımda maliyet çarpanı	1.15
MaxCount	Maksimum adet (0 = sınırsız)	0

Başlatma

```
_generatorSystem.Initialize(
    configs:      ConfigRegistry.Generators,
    economy:      _economyService,
    saveNotifier: _saveService
);
```

API

```
bool ok      = _generatorSystem.TryPurchase("coal-mine");
int count    = _generatorSystem.GetCount("coal-mine");
double cost   = _generatorSystem.GetNextCost("coal-mine");
double totalYps = _generatorSystem.CalculateTotalYield();
```

```
GeneratorSystem.OnGeneratorPurchased += (id) => RefreshUI();
```

Toplu Satın Alma

```
// 10 adet satın al
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Ten);

// Maksimum satın al
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Max);

// Belirli adete kadar satın al
var result = _generatorSystem.TryPurchaseBulk("coal-mine", BulkPurchaseMode.Until,
    untilCount: 25);

if (result.Purchased > 0)
    Debug.Log($"{result.Purchased} adet alındı, {result.TotalCost:F0} altın harcandı");
```

Gelir Formülü

gelir = BaseYieldPerSecond × adet × upgrade_çarpanı × prestige_çarpanı

upgrade_çarpanı UpgradeApplicationSystem tarafından StatType.GeneratorSpeed bazında hesaplanır.

9. PassiveIncomeService

Ne yapar: TickEngine'e abone olarak GeneratorSystem'in hesapladığı geliri her tick'te EconomyService'e ekler.

Başlatma

```
_passiveIncomeService.Initialize(  
    generators: _generatorSystem,  
    economy:    _economyService,  
    gameFlow:   _gameFlowStateMachine // null olabilir  
);
```

Her tick: $\text{CalculateTotalYield()} \times dt \rightarrow \text{EconomyService.AddResources()}.$

10. OfflineProgressService

Ne yapar: Oyun kapalıyken geçen süreyi hesaplar ve karşılık gelen pasif geliri oyun açılınca ekler.

EconomyConfigSO İlgili Alanlar

Alan	Açıklama	Varsayılan
OfflineCapHours	Maksimum offline kazanç süresi	8
OfflineYieldMultiplier	Offline gelir verimliliği	0.5

Başlatma

```
_offlineProgress.Initialize(  
    economy:    _economyService,  
    generators: _generatorSystem,  
    saveService: _saveService  
);  
// SaveService.OnSaveLoaded event'ine otomatik abone olur
```

UI'da Gösterme

```
OfflineProgressService.OnOfflineGainCalculated += (gold, seconds) =>  
{  
    if (gold > 0)  
        offlinePopup.Show(  
            $"{FormatGold(gold)} kazandınız",  
            $"({FormatTime(seconds)} çevrimdışydınız)"  
        );  
};
```

11. ClickYieldService

Ne yapar: Oyuncunun ekrana her tıkladığında aldığı anlık altın miktarını hesaplar. Basit clicker oyunları için kullanılır.

Not: Sahne içinde ClickLoopService da varsa ikisini aynı anda kullanmayın — Bootstrap otomatik uyarı verir ve ClickYieldService'i devre dışı bırakır. Birini seçin.

Başlatma

```
_clickYieldService.Initialize(  
    config:          myClickSourceConfig,    // ClickSourceConfig50  
    economy:         _economyService,  
    passiveYieldGetter: null                // YieldRateClickFraction için opsiyonel  
);  
_clickYieldService.SetInputProvider(_inputProvider);
```

API

```
// Tıklama gelirini işle (buton OnClick'e bağlayın)  
_clickYieldService.ProcessClick();  
  
// Tıklama başına kazanç (UI için)  
double perClick = _clickYieldService.GetClickYield();
```

12. ClickLoopService

Ne yapar: Dünyada yerleştirilen `ClickTarget` nesnelere tıklanarak hasar verilir; hedef yok edilince altın ödülü verilir ve belirli süre sonra yeniden doğar. Combo çarpanı, kritik vuruş ve oto-tıklama içerir.

Sistem Bileşenleri

Dosya	Görev
<code>ClickLoopConfigSO</code>	Combo/crit/oto-tıklama parametreleri
<code>ClickTargetConfigSO</code>	Tek bir hedef türünün HP/yield/respawn değerleri
<code>ClickTarget</code>	Sahnedeki tıklanabilir nesne (<code>MonoBehaviour</code>)
<code>ClickTargetRegistry</code>	Sahnedeki tüm aktif hedeflerin listesi
<code>ClickLoopService</code>	Ana servis; tıklamaları algılar, hasar uygular, gelir verir
<code>ClickLoopOfflineCalculator</code>	Çevrimdışı süre için oto-tıklama geliri hesabı
<code>ClickLoopHUDController</code>	Combo çubuğu, crit flash, HP barı

Config: ClickLoopConfigSO

Create → Endless Engine → Click Loop → Click Loop Config

Alan	Açıklama	Örnek
<code>ComboDecayDelay</code>	Sonraki tıklamaya kadar bekleme süresi (saniye)	1.5
<code>ComboDecayRate</code>	Saniyede azalan combo puanı	8
<code>MaxComboMultiplier</code>	Maksimum combo çarpanı	8
<code>ComboPointsPerStep</code>	Çarpan başına gereken combo puanı	5
<code>BaseCritChance</code>	Temel kritik şans (0-1)	0.05
<code>BaseCritMultiplier</code>	Kritik hasar çarpanı	3
<code>BaseAutoClickRate</code>	Saniyede otomatik tıklama sayısı	0
<code>OfflineCapHours</code>	Maks. offline hesap süresi	8
<code>OfflineEfficiency</code>	Offline gelir verimliliği	0.25

Config: ClickTargetConfigSO

Create → Endless Engine → Click Loop → Click Target Config

Alan	Açıklama	Örnek
<code>TargetId</code>	Benzersiz ID	"coin-bag"
<code>MaxHP</code>	Hedefin maksimum canı	10
<code>DamagePerClick</code>	Tıklama başına temel hasar	1
<code>BaseYield</code>	Yok edilince verilen temel altın	50
<code>AwardYieldPerClick</code>	Her tıklamada kısmi altın ver	false
<code>RespawnSeconds</code>	Yeniden doğma süresi	5
<code>ComboContribution</code>	Bu tıklamanın combo'ya katkısı	1
<code>DestructionVFXPrefab</code>	Yok edilince oynatılacak efekt	_(opsiyonel)_

ClickTarget Sahneye Ekleme

1. Yeni bir `GameObject` oluşturun.
2. `ClickTarget` component'ini ekleyin (`BoxCollider2D` otomatik eklenir — `[RequireComponent]` garantisi).

3. Inspector'da `_config` alanına `ClickTargetConfigSO` asset'ini atayın.
4. `GameObject`'e uygun bir Layer atayın (ör. `ClickableTargets`).

Hierarchy'de:

```
CoinBag
├─ Sprite (görsel çocuk)
├─ BoxCollider2D (otomatik)
└─ ClickTarget (siz eklersiniz)
```

Toplu Yerleştirme: ClickTargetSpawner

Birden fazla hedefi otomatik oluşturmak için:

```
// ClickTargetSpawner component'i
[SerializeField] private SpawnPoint[] _spawnPoints;

// SpawnPoint yapısı
[Serializable]
public struct SpawnPoint
{
    public ClickTargetConfigSO Config;
    public Vector3 WorldPosition;
}
```

`SpawnPoints` listesini Inspector'da doldurun; `Start()`'ta otomatik spawn eder.

Başlatma

```
_clickLoopService.Initialize(
    config: _clickLoopConfig,
    economy: _economyService,
    input: _inputProvider,
    targetLayer: LayerMask.GetMask("ClickableTargets"),
    statistics: _statisticsService, // opsiyonel
    vfx: _vfxController // opsiyonel
);
```

Olaylar

```
// Hedefe tıklandı
_clickLoopService.OnTargetClicked += (target, damage, gold, wasCrit) =>
{
    if (wasCrit) PlayCritEffect();
    ShowGoldPop(gold, target.WorldPosition);
};

// Hedef yok edildi
_clickLoopService.OnTargetDestroyed += (target) =>
{
    PlayDestroyEffect(target.WorldPosition);
};

// Combo değişti
_clickLoopService.OnComboChanged += (multiplier) =>
{
    comboText.text = $"x{multiplier:F1}";
};

// Kritik vuruş
_clickLoopService.OnCrit += (critMult) =>
{
    StartCritFlash();
};
```

HUD Bağlantısı

```
_clickLoopHUD.Initialize(_clickLoopService, _clickLoopConfig);
```

`ClickLoopHUDController` Inspector alanları:

Alan	Görev
_comboBar	Combo doluluk çubuğu (Slider/Image)
_comboText	"x2.5" gibi metin
_critFlash	Kritik vuruşta yanıp sönen Image
_targetHPBar	Aktif hedefin HP barı
_yieldPopText	Kazanılan altın pop-up metni

Offline Hesaplama

```
// Bootstrap'te:
_clickLoopOfflineCalc.Initialize(_clickLoopConfig, _economyService, _statisticsService);
_saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;

// OnDestroy'da:
_saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
```

Upgrade Bağlantıları (StatType Enum)

UpgradeNodeConfigSO'da AffectedStat olarak şu değerleri kullanabilirsiniz:

StatType	Etkisi
ClickDamage	Tıklama başına hasar
ClickTargetMaxHP	Hedef maksimum canı
ClickTargetRespawnRate	Yeniden doğma hızı
ClickYieldMultiplier	Altın kazanç çarpanı
ClickComboMultiplier	Maksimum combo çarpanı
ClickComboDecayRate	Combo azalma hızı
ClickCritChance	Kritik şans artışı
ClickCritMultiplier	Kritik hasar çarpanı
ClickAutoRate	Otomatik tıklama hızı

Kayıt / Yükleme

```
// Bootstrap'te:
_saveService.RegisterStateProvider(_clickLoopService);
// ClickLoopService otomatik olarak OnBeforeSave/OnAfterLoad yönetir.
// Her ClickTarget'ın respawn durumu ve timer'ı SaveData.ClickLoopState içinde saklanır.
```

13. HarvestLoopService

Ne yapar: Oyuncunun cursoru/parmağı dünya nesnelerinin (HarvestNode) üzerinde tutularak devamlı hasar verilir. Cursoru bir alanda gezdirmeye, tüm temas eden node'ları eşzamanlı hasar verme, combo ve çevrimdışı ilerleme içerir.

Sistem Bileşenleri

Dosya	Görev
HarvestAreaConfigSO	Cursor yarıçapı, tick hızı, combo parametreleri
HarvestNodeConfigSO	Tek bir node türünün HP/yield/respawn değerleri
HarvestNode	Sahnedeki hasat edilebilir nesne (MonoBehaviour)
HarvestNodeRegistry	Sahnedeki tüm aktif node'ların listesi
HarvestCursor	Cursor konumunu takip eden ve alan tespitini yapan servis
HarvestLoopService	Ana servis; tick'te hasar uygular, gelir verir
HarvestOfflineCalculator	Çevrimdışı süre için hasat geliri hesabı
HarvestHUDController	Combo çubuğu, yield pop, cursor radius göstergesi

Config: HarvestAreaConfigSO

Create → Endless Engine → Harvest → Harvest Area Config

Alan	Açıklama	Örnek
BaseRadius	Cursor'ın başlangıç etki yarıçapı (dünya birimi)	1.5
BaseTickInterval	Kaç saniyede bir hasar tick'i	0.25
ComboDecayDelay	Sonraki hit'e kadar bekleme süresi	1.5
ComboDecayRate	Saniyede azalan combo puanı	8
MaxComboMultiplier	Maksimum combo çarpanı	8
ComboPointsPerStep	Çarpan başına gereken puan	5
OfflineCapHours	Maks. offline hesap süresi	8
OfflineEfficiency	Offline gelir verimliliği	0.3

Config: HarvestNodeConfigSO

Create → Endless Engine → Harvest → Harvest Node Config

Alan	Açıklama	Örnek
NodeId	Benzersiz ID	"gold-ore"
MaxHP	Node'un maksimum canı	20
DamagePerTick	Tick başına temel hasar	2
BaseYield	Tüketilince verilen temel altın	100
YieldPerTickRatio	Her tick'te BaseYield'in kaçta biri verilir	0.05
RespawnSeconds	Yeniden doğma süresi	10
ComboContribution	Her hit'in combo'ya katkısı	1
DestructionVFXPrefab	Tüketilince oynatılacak efekt	_(opsiyonel)_

HarvestNode Sahneye Ekleme

1. Yeni bir GameObject oluşturun.
2. HarvestNode component'ini ekleyin (Collider2D otomatik eklenir).

3. Inspector'da `_config` alanına `HarvestNodeConfigSO` asset'ini atayın.
4. Node'u uygun bir Layer'a atayın (ör. `HarvestNodes`).

Hierarchy'de:

```
GoldOre
├─ Sprite (görsel çocuk)
├─ CircleCollider2D (otomatik)
└─ HarvestNode (siz eklersiniz)
```

HarvestCursor Kurulumu

1. Sahnede bir `GameObject` oluşturun.
2. `HarvestCursor` component'ini ekleyin.
3. Inspector'da `_config` (`HarvestAreaConfigSO`) ve `_harvestLayer` (`LayerMask`) atayın.
4. Bootstrap'te `InputProvider`'ı enjekte edin:

```
_harvestCursor.Inject(_inputProvider);
```

`HarvestCursor` her frame cursor dünya konumunu alır ve `Physics2D.OverlapCircleNonAlloc` ile temas eden node'ları tespit eder.

Başlatma

```
_harvestLoopService.Initialize(
    cursor: _harvestCursor,
    config: _harvestAreaConfig,
    economy: _economyService,
    statistics: _statisticsService, // opsiyonel
    vfx: _vfxController // opsiyonel
);
```

Olaylar

```
// Node'a hasar verildi
_harvestLoopService.OnNodeDamaged += (node, damage, gold) =>
{
    ShowGoldPop(gold, node.WorldPosition);
};

// Node tükendi
_harvestLoopService.OnNodeDepleted += (node) =>
{
    PlayDepletionEffect(node.WorldPosition);
};

// Gelir ödendi (tick bazında toplam)
_harvestLoopService.OnYieldAwarded += (totalGold) =>
{
    // Tick başına toplam kazanç
};

// Combo değişti
_harvestLoopService.OnComboChanged += (multiplier) =>
{
    comboBar.fillAmount = (multiplier - 1f) / (maxCombo - 1f);
};
```

HUD Bağlantısı

```
_harvestHUD.Initialize(_harvestLoopService, _harvestCursor, _harvestAreaConfig);
```

`HarvestHUDController` Inspector alanları:

Alan	Görev
<code>_comboBar</code>	Combo doluluk çubuğu
<code>_comboText</code>	"x2.5" metni
<code>_yieldPopText</code>	Kazanılan altın pop-up

Alan	Görev
_cursorRadiusIndicator	Cursor'ın etki alanını gösteren daire UI
_pxPerWorldUnit	UI→Dünya birimi dönüşüm katsayısı

Offline Hesaplama

```
// Bootstrap'te:
_harvestOfflineCalc.Initialize(_harvestAreaConfig, _economyService, _statisticsService);
_saveService.OnSaveLoaded += _harvestOfflineCalc.HandleSaveLoaded;

// OnDestroy'da:
_saveService.OnSaveLoaded -= _harvestOfflineCalc.HandleSaveLoaded;
```

Upgrade Bağlantıları (StatType Enum)

StatType	Etkisi
HarvestRadius	Cursor etki yarıçapı
HarvestTickRate	Tick hızı
HarvestYieldMultiplier	Altın kazanç çarpanı
HarvestNodeMaxHP	Node maksimum canı
HarvestNodeRespawnRate	Node yeniden doğma hızı
HarvestComboMultiplier	Maksimum combo çarpanı
HarvestComboDecayRate	Combo azalma hızı
HarvestMultiNodeBonus	Aynı anda birden fazla node vurmanın bonusu

Kayıt / Yükleme

```
// Bootstrap'te:
_saveService.RegisterStateProvider(_harvestLoopService);
// Her HarvestNode'un respawn durumu SaveData.HarvestState içinde saklanır.
```

ClickLoop ile HarvestLoop Karşılaştırması

Özellik	ClickLoop	HarvestLoop
Tetikleyici	Tıklama/dokunma	Cursor üzerinde tutma
Hedef	Tek nesne	Alan içindeki tüm nesneler
Hasar zamanlaması	Anlık	Tick bazlı
Zorluk	Aktif tıklama gerekir	Sürükleme/tutma
İdeal tür	Mobil tap, masaüstü click	Masaüstü mouse, tablet drag

14. UpgradeTreeService

Ne yapar: DAG (yönlü asiklik çizge) tabanlı yükseltme ağacı. Önkoşullu düğümler, rank sistemi, ağırlıklı kart çekimi.

Config: UpgradeNodeConfigSO

Create → Endless Engine → Upgrade Node Config

Alan	Açıklama	Örnek
NodeId	Benzersiz ID	"dmg-1"
DisplayName	Görüntülenecek ad	"Hasar I"
BaseCost	Temel altın maliyeti	100
CostScalingFactor	Her rank'ta maliyet çarpanı	1.5
MaxRank	Maksimum seviye (0 = sınırsız)	5
AffectedStat	Etkilediği stat (StatType enum)	Damage
EffectPerRank	Rank başına etki	0.1
EffectType	FlatBonus veya PercentBonus	PercentBonus
PrerequisiteNodeIDs	Önce açılması gereken düğümler	["dmg-basic"]
PrestigeGateRequirement	Erişim için gereken prestige sayısı	0
SelectionWeight	Kart çekimindeki görünme olasılığı	10

Tüm StatType Değerleri

Bu enum değerleri UpgradeNodeConfigSO.AffectedStat alanında kullanılır:

Savaş: Damage · AttackInterval · AttackRange · CritChance · CritMultiplier · AreaDamage

Sağlık: MaxHP · MoveSpeed · DamageReduction · HPRegen

Ekonomi: GoldDropMultiplier · GoldPickupRange · BonusRunReward · ComboMultiplier

Üretim: IdleYieldRate · GeneratorSpeed · OfflineYieldRate · ActiveRunPassiveBonus

Prestige: PrestigeMultiplier · StartingGoldBonus · RunDurationBonus · DoubleGeneratorChance

Hasat Loopuna Özel (8 adet): HarvestRadius · HarvestTickRate · HarvestYieldMultiplier · HarvestNodeMaxHP · HarvestNodeRespawnRate · HarvestComboMultiplier · HarvestComboDecayRate · HarvestMultiNodeBonus

Click Loopuna Özel (9 adet): ClickDamage · ClickTargetMaxHP · ClickTargetRespawnRate · ClickYieldMultiplier · ClickComboMultiplier · ClickComboDecayRate · ClickCritChance · ClickCritMultiplier · ClickAutoRate

Başlatma

```
_upgradeTree.Initialize(); // ConfigRegistry.Upgrades'i okur
_saveService.RegisterStateProvider(_upgradeTree);
```

API

```
// Rank sorgula
int rank = _upgradeTree.GetNode("dmg-1").CurrentRank;

// Satın alınabilir mi
bool canBuy = _upgradeTree.IsNodeAvailable("dmg-1");

// Maliyet
long cost = _upgradeTree.GetNodeCost("dmg-1");

// Satın al (EconomyService.TryPurchase içinden otomatik çağrılır)
_economyService.TryPurchase("dmg-1");

// Kart çekimi (dalga sonu ekran için)
var cards = _upgradeTree.GetAvailableNodes(); // tüm açık düğümler
```

Görsel Düzenleme

Tools → Endless Engine → Upgrade Tree Editor

15. UpgradeApplicationSystem

Ne yapar: Tüm stat hesaplarının merkezi. $\text{base_stat} \times (1 + \Sigma \text{Percent}) \times \text{PermanentMult}$ formülüyle efektif değeri döndürür.

Stat Formülü

$\text{efektif} = (\text{base} + \Sigma \text{Flat}) \times (1 + \Sigma \text{Percent}) \times \text{prestige_mult} \times \text{ascension_cascade}$

API

```
float damage = UpgradeApplicationSystem.GetEffectiveStat(StatType.Damage);  
float radius = UpgradeApplicationSystem.GetEffectiveStat(StatType.HarvestRadius);  
float crit   = UpgradeApplicationSystem.GetEffectiveStat(StatType.ClickCritChance);
```

Tüm sistemler (HarvestLoopService, ClickLoopService, vb.) bu API'yi kullanarak güncel stat değerlerini okur.

16. SkillTreeService

Ne yapar: Prestige ile kazanılan beceri puanları karşılığı açılan kalıcı beceri ağacı.

Config: SkillTreeConfigSO

Create → Endless Engine → Skill Tree Config

Her node: NodeId, DisplayName, SkillPointCost, AffectedStat, EffectPerRank, PrerequisiteNodeIDs.

Başlatma

```
_skillTree.Initialize(  
    configs:      ConfigRegistry.SkillTreeConfigs,  
    startingPoints: 0  
);  
_saveService.RegisterStateProvider(_skillTree);
```

API

```
int  points = _skillTree.SkillPoints;  
bool ok     = _skillTree.TryUnlock("skill-crit-1");  
  
IReadOnlyList<SkillEffect> effects = _skillTree.GetActiveEffects();
```

Görsel Düzenleme

Tools → Endless Engine → Skill Tree Editor

17. ResearchService

Ne yapar: Zaman bazlı araştırma kuyruğu. Araştırma başlatılır, X saniye/tick sonra tamamlanır ve kalıcı bonus aktive edilir.

Config: ResearchTreeConfigSO ve ResearchNodeConfigSO

Create → Endless Engine → Research → Research Tree Config
Create → Endless Engine → Research → Research Node Config

Alan	Açıklama
ResearchId	Benzersiz ID
DurationTicks	Kaç tick sürer
Cost	Altın maliyeti
AffectedStat	Tamamlanınca etkilenen stat
EffectValue	Etki miktarı

Başlatma

```
_researchService.Initialize(
    trees:          ConfigRegistry.ResearchTrees,
    economyService: _economyService,
    currencyService: null // ikincil para birimi için opsiyonel
);
_tickEngine.OnTick += _researchService.OnTick;
_saveService.RegisterStateProvider(_researchService);
```

API

```
// Kuyruğa ekle
bool ok = _researchService.EnqueueNode("tech-tree", "fire-upgrade");

// Tamamlandı mı
bool done = _researchService.IsCompleted("tech-tree", "fire-upgrade");

// Kuyrukta mı
bool queued = _researchService.IsQueued("tech-tree", "fire-upgrade");

// Olaylar
ResearchService.OnNodeCompleted += (treeId, nodeId) =>
    ShowToast($"Araştırma tamamlandı: {nodeId}");

ResearchService.OnResearchProgress += (treeId, nodeId, done, total) =>
    progressBar.fillAmount = (float)done / total;
```

18. PrestigeStateManager

Ne yapar: "Yumuşak sıfırlama" sistemi. Altın, dalga ilerlemesi ve yükseltmeleri sıfırlar; karşılığında kalıcı gelir çarpanı kazanılır.

Config: PrestigeConfigSO

Create → Endless Engine → Prestige Config

Alan	Açıklama	Örnek
MinWaveToPrestige	Prestige için gereken min dalga	10
BaseMultiplierPerPrestige	Prestige başına kalıcı çarpan artışı	0.1
MultiplierFormula	Linear / Exponential / Logarithmic	Linear
MaxPermanentMultiplier	Maksimum kalıcı çarpan (0 = sınırsız)	0

Başlatma

```
_prestigeManager.Initialize(
    economy: _economyService,
    waveManager: _waveSpawnManager,
    upgradeTree: _upgradeTree,
    saveService: _saveService
);
_saveService.RegisterStateProvider(_prestigeManager);
```

Prestige Akışı

```
TryPrestige() çağrılır
→ CanPrestige() kontrolü (min dalga, prestige gate)
→ Save-1 (çift güvenlik için)
→ OnPrestigeStarted event'i → tüm sistemler sıfırlanır
→ PrestigeCount artar, kalıcı çarpan hesaplanır
→ UpgradeApplicationSystem.SetPermanentMultiplier() çağrılır
→ OnPrestigeComplete(count, multiplier) event'i
→ Save-2
```

API

```
bool ok      = _prestigeManager.TryPrestige();
int count    = _prestigeManager.PrestigeCount;
bool canDo   = _prestigeManager.CanPrestige;
float mult    = _prestigeManager.GetPermanentMultiplier();

PrestigeStateManager.OnPrestigeStarted += () => ShowAnimation();
PrestigeStateManager.OnPrestigeComplete += (count, mult) => ShowReward(mult);
PrestigeStateManager.OnRealmUnlocked += (slug) => UnlockRealmUI(slug);
```

19. AscensionStateManager

Ne yapar: "Derin sıfırlama" — prestige'in de sıfırlandığı meta katman. Birden fazla Ascension Layer destekler.

Config: AscensionLayerConfigSO

Create → Endless Engine → Ascension Layer Config

Alan	Açıklama	Örnek
LayerIndex	Katman numarası (1'den başlar)	1
RequiredPrestigeCount	Bu katman için gereken prestige sayısı	10
PermanentMultiplierPerAscension	Her ascension'da kazanılan çarpan	2.0

Başlatma

```
_ascensionManager.Initialize(  
    prestigeManager: _prestigeManager,  
    database:        _ascensionDatabase,  
    saveService:     _saveService  
);  
_saveService.RegisterStateProvider(_ascensionManager);
```

API

```
bool ok      = _ascensionManager.TryAscend(layerIndex: 1);  
int count   = _ascensionManager.GetCount(layerIndex: 1);  
float cascade = _ascensionManager.GetCascadeMultiplier();
```

20. WaveSpawnManager

Ne yapar: Düşman dalgalarını yönetir. Her dalga düşmanları spawn eder; tamamlandığında sonraki dalgaya geçer.

Config: WaveConfigSO

Create → Endless Engine → Wave Config

Alan	Açıklama	Örnek
TotalWavesPerRun	Toplam dalga sayısı	30
WaveTransitionDelaySeconds	Dalgalar arası bekleme	2.0
UpgradeSelectionWaveInterval	Kaç dalgada bir upgrade kart ekranı	5
EnemiesPerWave	Dalga başına düşman sayısı (AnimationCurve)	—

Başlatma

```
_waveManager.Initialize(
    enemyManager: _enemyManager,
    config:        ConfigRegistry.Wave
);
_saveService.RegisterStateProvider(_waveManager);
```

API

```
_waveManager.StartWave();
int wave = _waveManager.CurrentWave;

WaveSpawnManager.OnWaveStarted    += (wave) => waveText.text = $"Dalga {wave}";
WaveSpawnManager.OnWaveCompleted += (wave) => ShowUpgradeScreen();
WaveSpawnManager.OnAllWavesCleared += ()    => ShowRunComplete();
```

21. AutoBattleController

Ne yapar: Otomatik savaş döngüsü. En yakın düşmanı hedef alır, saldırı aralığına göre hasar verir.

Config: PlayerBaseStatConfigSO

Create → Endless Engine → Player Base Stat Config

Alan	Açıklama
BaseAttackDamage	Temel hasar
BaseAttackInterval	Saldırı aralığı (saniye)
BaseMaxHP	Maksimum can
BaseCritChance	Kritik şans (0-1)
BaseCritMultiplier	Kritik çarpan

Başlatma

```
_abc.Initialize(  
    enemyManager: _enemyManager,  
    waveManager: _waveSpawnManager,  
    statProvider: _upgradeApplicationSystem,  
    playerConfig: ConfigRegistry.Player,  
    waveConfig: ConfigRegistry.Wave,  
    playerId: 1  
);  
_abc.SetInputProvider(_inputProvider);  
_abc.StartCombat();
```

22. DamageSystem

Ne yapar: Tüm hasar olaylarının işlendiği bus. Kritik hesabı, zırh azaltma, hasar türü ayrımı.

Dinleme

```
DamageSystem.OnDamageResolved += (hit) =>
{
    // hit.TargetID, hit.FinalDamage, hit.IsCritical, hit.Position
    SpawnDamageText(hit.Position, hit.FinalDamage, hit.IsCritical);
};
```

23. InputProvider

Ne yapar: Unity Input System'ı soyutlar. Tüm oyun kodu IInputProvider arayüzü üzerinden giriş okur.

IInputProvider Arayüzü

```
public interface IInputProvider
{
    Vector2 GetMoveVector();           // WASD / analog
    bool GetConfirmPressed();          // Space / A
    bool GetCancelPressed();           // Esc / B
    bool GetPausePressed();            // Esc / Start
    Vector2 GetMouseWorldPosition();    // fare dünya pozisyonu
    bool GetPointerClickedThisFrame(); // sol tık (bu frame)
    event Action OnPausePressed;        // debounce'lu pause eventi
}
```

Kurulum

1. Player GameObject'ine PlayerInput + InputProviderUnity ekleyin.
2. GameInputActions.inputactions asset'ini PlayerInput'a atayın.
3. Bootstrap'te `_inputProvider = playerGO.GetComponent<InputProviderUnity>()`.

Kural: Tek Dosya

InputProviderUnity.cs dışındaki hiçbir dosya UnityEngine.InputSystem namespace'ini import etmez. Oyun kodu sadece IInputProvider kullanır.

24. BuildingService

Ne yapar: Grid tabanlı bina yerleştirme. Binalar belirli hücrelere yerleştirilir, her biri pasif bonus sağlar.

Config: BuildingConfigSO

Create → Endless Engine → Building Config

Alan	Açıklama
BuildingId	Benzersiz ID
GridSize	Kapladığı hücre boyutu (Vector2Int)
BuildCost	Yapım maliyeti
UpgradeCosts[]	Tier başına yükseltme maliyeti
ProductionBonus	Sağladığı gelir bonusu

Kullanım

```
var result = _buildingService.TryPlace("farm", gridX: 2, gridY: 3);
if (result.Success)
    gridView.Render(result.Instance);

bool upgraded = _buildingService.TryUpgrade(instanceId);
_buildingService.Remove(instanceId);

BuildingService.OnBuildingPlaced += (inst) => UpdateGrid();
BuildingService.OnBuildingUpgraded += (inst) => UpdateGrid();
BuildingService.OnBuildingRemoved += (id) => UpdateGrid();
BuildingService.OnBuildingProduced += (id, amount) => ShowProduction(amount);
```


25. PetService

Ne yapar: Yoldaş sistemi. Her pet pasif bonus ve aktif yetenek sağlar; kilitleme, seviye atlatma, evrimleşme destekler.

Config: PetConfigSO

Create → Endless Engine → Pet Config

Alan	Açıklama
PetId	Benzersiz ID
DisplayName	Gösterim adı
AffectedStat	Etkilediği stat
BaseEffect	Temel etki değeri
LevelUpCosts[]	Seviye başına maliyet
EvolveAtLevel	Evrin için gereken seviye
EvolvesToPetId	Evrin sonucu pet ID'si

Başlatma

```
_petService.Initialize(  
    configs: ConfigRegistry.PetConfigs,  
    economy: _economyService  
);  
_saveService.RegisterStateProvider(_petService);
```

API

```
bool ok = _petService.TryEquip("fire-cat");  
bool lvl = _petService.TryLevelUp("fire-cat");  
bool evo = _petService.TryEvolve("fire-cat");  
int level = _petService.GetLevel("fire-cat");  
  
PetService.OnPetEquipped += (cfg) => ShowPetIcon(cfg);  
PetService.OnPetLeveledUp += (cfg, level) => ShowLevelUp(level);  
PetService.OnPetEvolved += (from, to) => ShowEvolution(to);
```

26. EventService

Ne yapar: Takvim tabanlı özel etkinlikler — belirli zaman dilimlerinde bonus çarpanlar, özel düşmanlar, sınırlı içerik aktive edilir.

Kullanım

```
ICollection<GameEvent> active = _eventService.GetActiveEvents();

EventService.OnEventStarted += (ev) => ShowEventBanner(ev.DisplayName);
EventService.OnEventEnded   += (ev) => HideEventBanner();
```

27. ChallengeService

Ne yapar: Opsiyonel kısıtlama modları. "Sadece 1 jeneratör", "saldırı yok ama 3× ödül" gibi zorluklar oyuna derinlik katar.

Config: ChallengeConfigSO

Alan	Açıklama
ChallengeId	Benzersiz ID
DisplayName	Görüntülenecek ad
Modifiers[]	Uygulanan kısıtlamalar (ChallengeModifier)
RewardMultiplier	Tamamlama ödül çarpanı
VictoryWave	Hedef dalga sayısı

Başlatma

```
_challengeService.Initialize(  
    economyService: _economyService,  
    upgradeTree:     _upgradeTree  
);
```

API

```
_challengeService.ActivateChallenge(config);  
_challengeService.CancelChallenge();  
  
bool  active = _challengeService.IsRunActive;  
bool  disabled = _challengeService.IsSystemDisabled("auto-battle");  
float reward = _challengeService.ActiveChallenge?.RewardMultiplier ?? 1f;  
  
ChallengeService.OnChallengeCompleted += (rewardGold) => ShowReward(rewardGold);  
ChallengeService.OnChallengeFailed    += ()           => ShowFailScreen();
```

28. MilestoneTracker

Ne yapar: Belirli eşiklere ulaşıncı ödül ve bildirim tetikler ("İlk 1000 altın", "10. prestige" vb.).

Config: MilestoneConfigSO

Alan	Açıklama
MilestoneId	Benzersiz ID
TrackedStat	Hangi değer izleniyor
Threshold	Eşik değeri
RewardType	Ödül türü
RewardValue	Ödül miktarı

Başlatma

```
_milestoneTracker.Initialize(  
    database:      _milestoneDatabase,  
    economyService: _economyService,  
    prestigeManager: _prestigeManager,    // opsiyonel  
    currencyService: _currencyService,    // opsiyonel  
    generatorSystem: _generatorSystem    // opsiyonel  
);  
_saveService.RegisterStateProvider(_milestoneTracker);
```

API

```
MilestoneTracker.OnMilestoneCompleted += (cfg) =>  
    ShowPopup($"Kilometre taşı: {cfg.DisplayName}!");
```

29. QuestService

Ne yapar: Kısa vadeli görevler — "100 düşman öldür", "3 jeneratör al" gibi günlük/haftalık hedefler.

Kullanım

```
ICollection<Quest> quests = _questService.GetActiveQuests();  
float progress           = _questService.GetProgress("kill-100");  
  
QuestService.OnQuestCompleted += (id) => GiveReward(id);  
QuestService.OnQuestRefreshed += ()   => RefreshQuestUI();
```

30. MinigameService

Ne yapar: Belirli aralıklarla tetiklenen aktif mini oyunlar. Başarıyla tamamlananlar anlık büyük ödül verir.

Başlatma

```
_minigameService.Initialize(_tickEngine, _economyService);
```

API

```
bool available = _minigameService.IsMinigameAvailable;  
_minigameService.StartMinigame();
```

```
MinigameService.OnMinigameCompleted += (reward) => ShowReward(reward);  
MinigameService.OnMinigameFailed    += ()      => ShowFailEffect();
```

31. MergeService

Ne yapar: Merge (birleştirme) mekaniği — 2 aynı tier'daki nesneyi birleştir, bir üst tier nesne elde et.

Config: MergeConfigSO

Her merge zinciri bir MergeConfigSO'da tanımlanır:

Create → Endless Engine → Merge Config

API

```
MergeResult result = _mergeService.TryMerge(itemA, itemB);

if (result.Success)
{
    gridView.Remove(itemA.Position);
    gridView.Remove(itemB.Position);
    gridView.Place(result.ResultItem, targetPos);
}

MergeService.OnMergeCompleted += (r) => PlayMergeVFX(r.ResultItem);
```

32. ConversionService

Ne yapar: Bir kaynak türünü diğerine dönüştürür (altın → kristal, jem → enerji vb.).

Config: ConversionDatabaseSO

Create → Endless Engine → Conversion Database

Her tarif: InputCurrency, InputAmount, OutputCurrency, OutputAmount.

API

```
bool ok = _conversionService.TryConvert("gold-to-gems", amount: 100);
```


33. TraitService

Ne yapar: Oyuncunun kalıcı nitelik seçimi. Prestige/Ascension'da sunulan trait'ler oyun stilini şekillendirir.

API

```
ICollection<ITrait> available = _traitService.GetAvailableTraits();  
_traitService.SelectTrait("speed-demon");  
  
float mult = _traitService.GetMultiplierForStat(StatType.AttackInterval);
```

34. UnlockLogService

Ne yapar: Kilidi açılan içeriklerin kayıtlı geçmişini tutar.

Başlatma

```
_unlockLogService.Initialize(ConfigRegistry.UnlockEntries);  
_saveService.RegisterStateProvider(_unlockLogService);
```

API

```
_unlockLogService.RecordUnlock("fire-realm", UnlockType.Realm);  
bool unlocked = _unlockLogService.IsUnlocked("fire-realm");  
IReadOnlyList<UnlockEntry> history = _unlockLogService.GetHistory();
```

35. StatisticsService

Ne yapar: Koşu ve ömür boyu istatistikleri toplar. "En yüksek dalga", "toplam tıklama", "kaç prestige yapıldı" vb.

Başlatma

```
_statisticsService.Initialize(ConfigRegistry.StatDefinitions);
_saveService.RegisterServiceProvider(_statisticsService);
```

API

```
// Artır
_statisticsService.Add("harvest.total_gold", goldAmount);
_statisticsService.Add("clickloop.total_targets_destroyed", 1);

// En yüksek güncelle (azalmaz)
_statisticsService.SetIfHigher("highest_wave", currentWave);

// Oku
double total = _statisticsService.Get("harvest.total_gold");

// Sistem sabit ID'leri:
// HarvestLoopService.StatIdTotalGold           = "harvest.total_gold"
// HarvestLoopService.StatIdTotalNodes           = "harvest.total_nodes_harvested"
// HarvestLoopService.StatIdBestCombo            = "harvest.best_combo_multiplier"
// ClickLoopService.StatIdTotalGold              = "clickloop.total_gold"
// ClickLoopService.StatIdTotalDestroyed         = "clickloop.total_targets_destroyed"
// ClickLoopService.StatIdBestCombo              = "clickloop.best_combo_multiplier"
```

StatDefinitionSO

Create → Endless Engine → Stat Definition

Alan	Açıklama
StatId	Benzersiz string ID
DisplayName	Kullanıcıya gösterilen ad
IsPeakValue	true = azalmaz (yüksek dalga), false = birikir (toplam altın)

36. NotificationService

Ne yapar: Sistem içi ve işletim sistemi bildirimlerini yönetir.

API

```
_notifService.Show("Araştırma tamamlandı!", NotificationType.Info);  
_notifService.Schedule("Enerji doldu!", delaySeconds: 3600);  
_notifService.CancelAll();
```

37. LeaderboardService

Ne yapar: Yerel ve uzak skor tabloları.

API

```
await _leaderboardService.SubmitScoreAsync("highest-wave", waveNumber);  
LeaderboardEntry[] entries = await _leaderboardService.FetchAsync("highest-wave", count:  
    10);
```

38. TutorialService

Ne yapar: Adım adım öğretici — koşul tabanlı adımlar sırayla ilerler.

API

```
_tutorialService.Initialize(_saveService);  
_tutorialService.TriggerStep("first-generator-buy");  
bool active = _tutorialService.IsStepActive("first-generator-buy");
```

```
TutorialService.OnStepStarted += (id) => ShowArrow(id);  
TutorialService.OnStepCompleted += (id) => HideArrow();
```

39. RealmSystem

Ne yapar: Farklı oyun bölgelerini yönetir. Her realm kendi config paketine sahiptir — düşman stats, ekonomi, görsel tema değişir.

Yeni Realm Oluşturma

Tools → Endless Engine → Content Pack Wizard → Realm adı gir → Create All

Bu işlem Assets/Configs/[RealmName]/ altında 9 SO + RealmPackSO oluşturur ve RealmRegistry'ye otomatik ekler.

Realm Değiştirme

```
await ConfigRegistry.BeginRealmSwapAsync(realmRegistry.GetPack("volcano"));

ConfigRegistry.OnRealmSwapped += () =>
{
    // Cache'leri yenile
    _resourceHardCap = ConfigRegistry.Economy.ResourceHardCap;
};
```

40. AudioService

Ne yapar: Pool tabanlı SFX çalma, müzik yönetimi, AudioManager snapshot geçişleri.

API

```
_audioService.PlaySFX(clip, volume: 1f, pitch: 1f);
_audioService.SetSFXVolume(0.8f); // PlayerPrefs'e kaydeder
_audioService.SetMusicVolume(0.5f);
_audioService.Duck(); // Müziği kısaltma (ör. dialog sırasında)
_audioService.Unduck();
```


41. BigDouble

Ne yapar: 1e308'i aşan büyük sayıları temsil eden özel sayı tipi. Standart double sınırını aşan idle ekonomiler için.

Ne Zaman Kullanılır?

Oyuncunun 100 saatte ulaşabileceği değer ~1.8e308'i aşacaksa BigDouble gerekir.

Aktifleştirme

EconomyConfigSO → NumberBackend → BigDouble. Tüm EconomyService API'si otomatik olarak BigDouble kullanır — kod değişikliği gerekmez.

Yapısı

değer = Mantissa × 10^{Exponent}

```
var bd = new BigDouble(1.5, 300); // 1.5 × 10300  
bd.Mantissa // 1.5  
bd.Exponent // 300  
bd.Format(BigDouble.Notation.Letter, 2) // "1.50aa"
```

Dikkat Edilecekler

- BigDouble(0.0, 999) → IsZero = true
- ToDouble() için exponent > 308 → Infinity döner; sadece görüntüleme için kullanın
- Backend değiştirirseniz mevcut kayıtlar için SaveMigration yazın

42. Editor Araçları

Tüm araçlar: Tools → Endless Engine → ...

Araç	Ne Yapar	Kısa Kullanım
New Game Wizard	Oyun tipine göre tüm temel config'leri otomatik oluşturur	Oyun tipi seç → Create
Content Pack Wizard	Yeni Realm için 9 SO + RealmPackSO + Registry kaydı	Realm adı gir → Create All
Upgrade Tree Editor	DAG upgrade ağacını görsel düzenleme	Düğüm sürükley, ok çiz
Skill Tree Editor	Skill ağacını görsel düzenleme	Aynı şekilde
Trait Tree Editor	Trait ağacını görsel düzenleme	Aynı şekilde
Generator Editor	Generator asset'lerini tablo şeklinde toplu düzenleme	Açık → Düzenle
Generator Asset Creator	Birden fazla GeneratorConfigSO'yu toplu oluşturur	İsimleri gir → Batch Create
ID Registry Window	Tüm SO ID'leri tarar; duplikat ve yetim ID'leri listeler	Scan → Jump ile git
Config Validator	Tüm config SO'ları doğrulama kurallarına göre tarar	Validate Configs
Economy Simulator	Gelir eğrisini simüle eder	Parametreleri gir → Simulate
Economy Tuning Window	Config değerlerini oyun açıkken canlı düzenle	Editor mod only
Schema Bump Utility	SchemaVersion artırır, migrasyon şablonu oluşturur	Bump Version

New Game Wizard Oyun Tipleri

Tip	Oluşturulan Sistemler
Pure Idle	Jeneratör + pasif gelir
Clicker Idle	Tıklama + jeneratör
Merge Idle	Merge sistemi + ekonomi
Prestige-Heavy	Tüm prestige katmanları
Wave Idle	AutoBattle + wave + upgrade
Incremental RPG	Tüm sistemler dahil

43. Test Stratejisi

Katman 1 — Unit Tests

Tek sistem, izole, bağımlılık yok. NUnit [Test], EditMode.

```
[Test]
public void AddResources_BelowCap_IncreasesBalance()
{
    var svc = new GameObject().AddComponent<EconomyService>();
    svc.InjectStateForTesting(currentResources: 100, hardCap: 1000, startingGold: 0);
    svc.AddResources(50);
    Assert.AreEqual(150, svc.CurrentResources, 0.01);
}
```

Katman 2 — Integration Tests

Birden fazla sistem birlikte. Assets/Tests/integration/ altında.

```
[Test]
public void OnBeforeSave_DestroyedTarget_WritesRespawnEntry()
{
    _target.ApplyDamage(_targetConfig.MaxHP); // hedefi yok et
    var save = new SaveData();
    save.EnsureDefaults();
    _service.OnBeforeSave(save);

    Assert.AreEqual(1, save.ClickLoopState.TargetStates.Count);
    Assert.IsTrue(save.ClickLoopState.TargetStates.First().Value.IsRespawning);
}
```

Katman 3 — Performance Tests

Kritik yollarda GC allocation ve frame süresi.

```
[Test, Performance]
public void AddResources_HotPath_ZeroAllocation()
{
    Measure.Method(() => _economy.AddResources(1.0f))
        .GC()
        .Run();
}
```

Test Çalıştırma

Window → General → Test Runner → EditMode → Run All

Önemli Test Kalıpları

ConfigRegistry her testte sıfırlanmalı:

```
[SetUp]    public void Setup()    => ConfigRegistry.InjectForTesting(economy: _cfg);
[TearDown] public void Teardown() => ConfigRegistry.ClearForTesting();
```

Input System testleri için:

```
Press(_keyboard.wKey);
InputSystem.Update(); // EditMode'da manuel güncelleme şart
Assert.AreEqual(1f, _provider.GetMoveVector().y, 0.05f);
```

MonoBehaviour testlerinde config enjeksiyonu:

```
_go = new GameObject();
_go.SetActive(false); // Awake'i ertele
_go.AddComponent<BoxCollider2D>();
_target = _go.AddComponent<ClickTarget>();
SetField(_target, "_config", _config); // reflection ile enjekte et
_go.SetActive(true); // şimdi Awake çalışır
```

```
static void SetField(object t, string n, object v)
{
    var f = t.GetType().GetField(n,
        BindingFlags.NonPublic | BindingFlags.Instance);
    f?.SetValue(t, v);
}
```

44. Tarif 1: Klasik Idle

Hedef: Cookie Clicker benzeri — tıkla, jeneratör al, yükseltme yap.

Kullanılan Sistemler

EconomyService · TickEngine · GeneratorSystem · PassiveIncomeService · ClickYieldService · UpgradeTreeService · SaveService · OfflineProgressService

Kurulum

Tools → New Game Wizard → Pure Idle → Create

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  GeneratorSystem
  PassiveIncomeService
  ClickYieldService
  UpgradeTreeService
  OfflineProgressService
```

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);
    await _saveService.LoadAsync();

    _upgradeTree.Initialize();
    _economyService.Initialize(_upgradeTree, _saveService);
    _generatorSystem.Initialize(ConfigRegistry.Generators, _economyService,
        _saveService);
    _passiveIncome.Initialize(_generatorSystem, _economyService, null);
    _clickYield.Initialize(null, _economyService, null);
    _clickYield.SetInputProvider(_inputProvider);
    _offline.Initialize(_economyService, _generatorSystem, _saveService);

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_generatorSystem);
}
```

Tıklama Butonu

```
public void OnClickGold() => _clickYield.ProcessClick();
```

Jeneratör UI

```
void RefreshUI()
{
    foreach (var cfg in ConfigRegistry.Generators)
    {
        var btn = GetButtonFor(cfg.GeneratorId);
        btn.countText.text = _generatorSystem.GetCount(cfg.GeneratorId).ToString();
        btn.costText.text =
            FormatGold(_generatorSystem.GetNextCost(cfg.GeneratorId));
        btn.ypsText.text = $"{_generatorSystem.GetYield(cfg.GeneratorId):F1}/s";
    }
}
```

Offline Popup

```
OfflineProgressService.OnOfflineGainCalculated += (gold, secs) =>
{
    if (gold > 0)
        offlinePopup.Show($"{FormatGold(gold)} kazandınız ({FormatTime(secs)}
            çevrimdışıydınız)");
};
```

45. Tarif 2: Aktif Clicker

Hedef: Ekranı yerleştirilen hedef nesnelere tıkla, yok et, altın kazan, yeniden doğmasını bekle.

Kullanılan Sistemler

EconomyService · TickEngine · ClickLoopService · ClickTarget · ClickTargetSpawner · ClickLoopOfflineCalculator · ClickLoopHUDController · UpgradeTreeService · StatisticsService · SaveService

Kurulum

Tools → New Game Wizard → Clicker Idle → Create

Sahne Yapısı

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  UpgradeTreeService
  StatisticsService
  ClickLoopService
  ClickLoopOfflineCalculator
  VFXController
Player/
  PlayerInput
  InputProviderUnity
World/
  CoinBag_1 [ClickTarget + BoxCollider2D]
  CoinBag_2 [ClickTarget + BoxCollider2D]
  CoinBag_3 [ClickTarget + BoxCollider2D]
UI/
  ClickLoopHUD [ClickLoopHUDController]
```

Config'leri Oluşturun

Create → Endless Engine → Click Loop → Click Loop Config (ComboDecayDelay=1.5, MaxComboMultiplier=8)
Create → Endless Engine → Click Loop → Click Target Config (TargetId="coin-bag", MaxHP=5, BaseYield=30)

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);
    await _saveService.LoadAsync();

    _upgradeTree.Initialize();
    _economyService.Initialize(_upgradeTree, _saveService);
    _statisticsService.Initialize(ConfigRegistry.StatDefinitions);

    _clickLoopService.Initialize(
        config:      _clickLoopConfig,
        economy:     _economyService,
        input:       _inputProvider,
        targetLayer: LayerMask.GetMask("ClickableTargets"),
        statistics:  _statisticsService,
        vfx:         _vfxController
    );

    _clickLoopHUD.Initialize(_clickLoopService, _clickLoopConfig);

    _clickLoopOfflineCalc.Initialize(_clickLoopConfig, _economyService,
        _statisticsService);
    _saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_statisticsService);
    _saveService.RegisterStateProvider(_clickLoopService);
}

private void OnDestroy()
{
    _saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
}
```

ClickTarget Sahnede Kurulumu

Her hedef GameObject için:

1. ClickTarget component ekle.
2. Inspector'da _config alanına ClickTargetConfigSO asset'ini ata.
3. Layer'ı ClickableTargets olarak ayarla.

Upgrade Yükseltmeleri

```
Create → Upgrade Node Config
NodeId = "click-dmg-1"
DisplayName = "Tıklama Hasarı I"
AffectedStat = ClickDamage
EffectPerRank = 0.2  (+%20 her rank)
MaxRank = 5
BaseCost = 50
```

UI

```
// Goldı göster
EconomyService.OnResourcesChanged += (current, _) =>
    goldText.text = FormatGold(current);

// Combo göster
_clickLoopService.OnComboChanged += (mult) =>
    comboText.text = $"x{mult:F1}";

// Kritik flaş
_clickLoopService.OnCrit += (_) =>
    StartCoroutine(CritFlashCoroutine());
```


46. Tarif 3: Hasat Loop

Hedef: Mouse'u/parmağı dünya nesneleri üzerinde sürükleyerek hasar ver, altın kazan.

Kullanılan Sistemler

EconomyService · TickEngine · HarvestLoopService · HarvestCursor · HarvestNode · HarvestNodeSpawner · HarvestOfflineCalculator · HarvestHUDController · UpgradeTreeService · StatisticsService · VFXController · SaveService

Sahne Yapısı

```
Bootstrap
Services/
  SaveService
  EconomyService
  TickEngine
  UpgradeTreeService
  StatisticsService
  HarvestLoopService
  HarvestOfflineCalculator
  VFXController
World/
  HarvestCursor [HarvestCursor + (ayrı G0)]
  Ore_1 [HarvestNode + CircleCollider2D]
  Ore_2 [HarvestNode + CircleCollider2D]
  Ore_3 [HarvestNode + CircleCollider2D]
Player/
  PlayerInput
  InputProviderUnity
UI/
  HarvestHUD [HarvestHUDController]
```

Config'leri Oluşturun

Create → Endless Engine → Harvest → Harvest Area Config
(BaseRadius=1.5, BaseTickInterval=0.25, ComboDecayDelay=1.5)

Create → Endless Engine → Harvest → Harvest Node Config
(NodeId="gold-ore", MaxHP=20, DamagePerTick=2, BaseYield=100, RespawnSeconds=10)

Bootstrap Script'i

```
private async void Awake()
{
    ConfigRegistry.InjectForTesting(economy: _economyConfig);
    await _saveService.LoadAsync();

    _upgradeTree.Initialize();
    _economyService.Initialize(_upgradeTree, _saveService);
    _statisticsService.Initialize(ConfigRegistry.StatDefinitions);

    _harvestCursor.Inject(_inputProvider);

    _harvestLoopService.Initialize(
        cursor: _harvestCursor,
        config: _harvestAreaConfig,
        economy: _economyService,
        statistics: _statisticsService,
        vfx: _vfxController
    );

    _harvestHUD.Initialize(_harvestLoopService, _harvestCursor, _harvestAreaConfig);

    _harvestOfflineCalc.Initialize(_harvestAreaConfig, _economyService,
        _statisticsService);
    _saveService.OnSaveLoaded += _harvestOfflineCalc.HandleSaveLoaded;

    _saveService.RegisterStateProvider(_economyService);
    _saveService.RegisterStateProvider(_upgradeTree);
    _saveService.RegisterStateProvider(_statisticsService);
    _saveService.RegisterStateProvider(_harvestLoopService);
}

private void OnDestroy()
{
    _saveService.OnSaveLoaded -= _harvestOfflineCalc.HandleSaveLoaded;
}
```

HarvestNode Sahnede Kurulumu

Her node GameObject için:

1. HarvestNode component ekle.
2. Inspector'da _config alanına HarvestNodeConfigSO asset'ini ata.
3. Layer'ı HarvestNodes olarak ayarla.

Upgrade Yükseltmeleri

```
Create → Upgrade Node Config
NodeId = "harvest-radius-1"
DisplayName = "Hasat Yarıçapı I"
AffectedStat = HarvestRadius
EffectPerRank = 0.15 (+%15 yarıçap artışı)
MaxRank = 5
```

47. Tarif 4: Idle-vs

Hedef: Otomatik savaş + dalga sistemi + upgrade kartı + prestige.

Kullanılan Sistemler

Tarif 1'deki her şey + WaveSpawnManager · AutoBattleController · DamageSystem · EnemyManager · HealthSystem · UpgradeApplicationSystem · PrestigeStateManager · InputProviderUnity

Ek Config'ler

Create → Wave Config (TotalWavesPerRun=30, WaveTransitionDelaySeconds=2)
 Create → Player Config (BaseAttackDamage=10, BaseMaxHP=100, BaseCritChance=0.05)
 Create → Prestige Config (MinWaveToPrestige=10, BaseMultiplierPerPrestige=0.1)

Ek Sistemler Bootstrap'e Ekle

```
_waveManager.Initialize(_enemyManager, ConfigRegistry.Wave);

_abc.Initialize(_enemyManager, _waveManager, _uas,
    ConfigRegistry.Player, ConfigRegistry.Wave, playerId: 1);
_abc.SetInputProvider(_inputProvider);
_abc.StartCombat();

_prestigeManager.Initialize(_economyService, _waveManager, _upgradeTree, _saveService);
_saveService.RegisterStateProvider(_prestigeManager);
_saveService.RegisterStateProvider(_waveManager);
```

Dalga Sonu Upgrade Ekranı

```
WaveSpawnManager.OnWaveCompleted += (wave) =>
{
    if (wave % ConfigRegistry.Wave.UpgradeSelectionWaveInterval == 0)
    {
        _abc.StopCombat();
        var nodes = _upgradeTree.GetAvailableNodes();
        // 3 tanesi rastgele seç:
        var cards = nodes
            .OrderBy(_ => UnityEngine.Random.value)
            .Take(3)
            .Select(n => n.Config)
            .ToArray();

        upgradeScreen.Show(cards, (chosen) =>
        {
            _economyService.TryPurchase(chosen.NodeId);
            upgradeScreen.Hide();
            _abc.StartCombat();
        });
    }
};
```

Prestige Butonu

```
void Update()
{
    prestigeBtn.interactable = _prestigeManager.CanPrestige;
    multiplierText.text = $"x{_prestigeManager.GetPermanentMultiplier():F2}";
}

public void OnPrestigeClick() => _prestigeManager.TryPrestige();
```

48. Tarif 5: Merge Idle

Hedef: Merge board + pasif gelir + ekonomi.

Kullanılan Sistemler

EconomyService · MergeService · GeneratorSystem · PassiveIncomeService · UpgradeTreeService · SaveService

Kilit Adımlar

```
// İki objeyi birleştir
public void OnItemDropped(MergeItem a, MergeItem b, Vector2Int pos)
{
    if (a.Tier != b.Tier) return;

    var result = _mergeService.TryMerge(a, b);
    if (!result.Success) return;

    gridView.Remove(a.Position);
    gridView.Remove(b.Position);
    gridView.Place(result.ResultItem, pos);
}

// Yüksek tier merge → anlık altın bonusu
MergeService.OnMergeCompleted += (r) =>
    _economyService.AddResources(r.ResultItem.Tier * 10.0);
```

49. Tarif 6: Prestige-Heavy RPG Idle

Hedef: Derin prestige + ascension katmanları + skill tree + trait sistemi + araştırma.

Kullanılan Sistemler

Tarif 4'teki her şey + AscensionStateManager · SkillTreeService · TraitService · ResearchService · MilestoneTracker · StatisticsService

Ascension Kurulumu

```
Create → Ascension Layer Config
  LayerIndex = 1
  RequiredPrestigeCount = 10
  PermanentMultiplierPerAscension = 2.0

_ascensionManager.Initialize(_prestigeManager, _ascensionDb, _saveService);
_saveService.RegisterStateProvider(_ascensionManager);

AscensionStateManager.OnAscensionComplete += (layer, count, cascade) =>
  Debug.Log($"Ascension L{layer} #{count} | Cascade: {cascade:F2}x");
```

Prestige → Skill Puanı

```
PrestigeStateManager.OnPrestigeComplete += (count, mult) =>
  _skillTree.AddPoints(1);
```

Ascension → Trait Seçimi

```
AscensionStateManager.OnAscensionStarted += () =>
{
  var traits = _traitService.GetAvailableTraits();
  traitScreen.Show(traits, chosen => _traitService.SelectTrait(chosen.TraitId));
};
```

Araştırma Kuyruğu

```
_researchService.Initialize(ConfigRegistry.ResearchTrees, _economyService, null);
_tickEngine.OnTick += _researchService.OnTick;
_saveService.RegisterStateProvider(_researchService);

public void QueueResearch(string treeId, string nodeId)
{
  if (_researchService.EnqueueNode(treeId, nodeId))
    researchUI.RefreshQueue();
}

ResearchService.OnNodeCompleted += (tree, node) =>
  ShowToast($"Araştırma tamamlandı: {node}");
```

50. Sorun Giderme

"ConfigNotLoadedException" veya null ref config hatası

Config'e OnConfigsLoaded tetiklenmeden önce erişiyorsunuz.

```
// Çözüm: Sisteminizi bu event'e abone edin
ConfigRegistry.OnConfigsLoaded += () => { Initialize(); };
```

Testlerde:

```
[SetUp] public void Setup() => ConfigRegistry.InjectForTesting(economy: _cfg);
[TearDown] public void Teardown() => ConfigRegistry.ClearForTesting();
```

Testlerde Input System her zaman 0 döndürüyor

EditMode [UnityTest]'te yield return null InputSystem.Update()'i tetiklemez.

```
// Çözüm: [Test] kullanın ve Press() sonrası manuel güncelleme yapın
[Test]
public void WKey_ReturnsUpVector()
{
    Press(_keyboard.wKey);
    InputSystem.Update(); // şart
    Assert.AreEqual(1f, _provider.GetMoveVector().y, 0.05f);
}
```

ClickTarget / HarvestNode Awake'te config null

Test ortamında SetActive(true) çağrısından önce config enjekte edilmemiş.

```
// Çözüm: Önce devre dışı oluştur, config'i enjekte et, sonra aktif et
_go = new GameObject();
_go.SetActive(false);
_go.AddComponent<BoxCollider2D>();
_target = _go.AddComponent<ClickTarget>();
SetField(_target, "_config", _config); // reflection
_go.SetActive(true); // şimdi Awake çalışır
```

ClickLoopService ve ClickYieldService aynı anda altın veriyor

Bootstrap otomatik olarak uyarır ve ClickYieldService'i devre dışı bırakır. Bunu kasıtlı olarak tetiklemek için:

- ClickYieldService kullanıyorsanız _clickLoopService alanını Inspector'da boş bırakın.
- ClickLoopService kullanıyorsanız _clickYieldService alanını boş bırakın.

Prestige sonrası stat güncellenmiyor

```
// Initialize() çağrısında UpgradeApplicationSystem bağlı mı kontrol edin:
_prestigeManager.Initialize(_economy, _wave, _upgradeTree, _save);
// UpgradeApplicationSystem, OnPrestigeComplete'e kendi içinde abone olur.
```

Çevrimdışı hesap çalışmıyor

SaveService.OnSaveLoaded olayına abone olduğundan ve HandleSaveLoaded imzasının (SaveData data, bool isNewGame) olduğundan emin olun:

```
_saveService.OnSaveLoaded += _clickLoopOfflineCalc.HandleSaveLoaded;
// OnDestroy'da:
_saveService.OnSaveLoaded -= _clickLoopOfflineCalc.HandleSaveLoaded;
```

BigDouble kayıt/yükleme uyumsuzluğu

EconomyConfigS0.NumberBackend değeri değiştirildiyse eski kayıtlar uyumsuz olur.

Tools → Endless Engine → Schema Bump Utility → Bump Version
// Oluşturulan MigrationV{N}.cs dosyasında NumberBackendName kontrolü ekleyin

HarvestNode veya ClickTarget kayıt sonrası senkronize değil

SaveService.RegisterStateProvider() Bootstrap'te çağırıldığından ve sıranın
SaveConstants.SaveProviderOrder.Harvest (88) / SaveConstants.SaveProviderOrder.ClickLoop (89) olduğundan
emin olun.

Endless Engine v1.1.0 — Kullanım Kılavuzu Sonu